



RAPPORT DE STAGE

Automatisation de la gestion des vulnérabilités dans les pipelines
MLOps

PAR
KANE ALPHA

Superviseur :
Jonathan Roy, Ph.D.
Professeur au Département d'informatique et de mathématique

RAPPORT PRÉSENTÉ À L'UNIVERSITÉ DU QUÉBEC À CHICOUTIMI COMME
EXIGENCE PARTIELLE EN VUE DE L'OBTENTION DU GRADE DE
MAÎTRISE EN INFORMATIQUE
OPTION CYBERSÉCURITÉ

QUÉBEC, CANADA

© KANE ALPHA, 2026

Dédicace

Je dédie ce travail à ma famille, pour leur soutien indéfectible, leur patience et leurs encouragements tout au long de mon parcours académique.

À mes proches, qui ont toujours cru en moi et m'ont motivé à persévérer malgré les défis rencontrés.

Enfin, à toutes les personnes qui m'ont accompagné de près ou de loin dans la réalisation de ce projet.

Remerciements

Je tiens à exprimer ma sincère reconnaissance envers mon superviseur de stage, Monsieur Jonathan Roy, professeur au Département d'informatique et de mathématique de l'Université du Québec à Chicoutimi, pour la qualité de son encadrement, la pertinence de ses conseils ainsi que sa disponibilité tout au long de ce projet.

Je souhaite également adresser mes remerciements à Madame Caroline Gagné, professeure au Département d'informatique et de mathématique, pour ses recommandations avisées, son soutien académique et l'intérêt qu'elle a porté à ce travail.

Mes remerciements s'adressent aussi à l'ensemble du corps professoral et du personnel du Département d'informatique et de mathématique pour la qualité de la formation offerte et l'encadrement fourni durant mon parcours de maîtrise.

Enfin, je tiens à remercier chaleureusement ma famille et mes proches pour leur soutien indéfectible, leur patience et leurs encouragements constants, qui ont grandement contribué à l'aboutissement de ce projet.

Avant-propos

Ce rapport s’inscrit dans le cadre du stage de maîtrise en informatique, option cybersécurité, réalisé à l’Université du Québec à Chicoutimi.

Ce stage avait pour objectif principal de mettre en pratique les connaissances acquises durant la formation, tout en développant des compétences avancées dans le domaine de la sécurité des systèmes d’intelligence artificielle.

Le projet réalisé porte sur l’automatisation de la gestion des vulnérabilités dans les pipelines MLOps, un domaine en pleine expansion à l’intersection de la cybersécurité et de l’intelligence artificielle.

Résumé

Dans un contexte où les systèmes d'intelligence artificielle sont de plus en plus intégrés dans les environnements de production, les pipelines MLOps jouent un rôle essentiel dans l'automatisation du cycle de vie des modèles. Toutefois, ces environnements introduisent de nouvelles vulnérabilités liées au code, aux dépendances, aux modèles et aux infrastructures.

Ce projet vise à concevoir et à implémenter une solution automatisée permettant de détecter, d'analyser et de gérer les vulnérabilités au sein des pipelines MLOps. Pour ce faire, un pipeline CI/CD sécurisé a été développé en intégrant plusieurs outils d'analyse tels que Bandit, Trivy et ModelScan.

Une approche centralisée a également été mise en œuvre à travers un module d'enrichissement basé sur la base de données NVD, permettant d'associer des identifiants CVE, des scores CVSS et des niveaux de sévérité aux vulnérabilités détectées.

Les résultats obtenus démontrent l'efficacité de l'approche proposée, notamment en termes d'automatisation, de couverture des risques et d'intégration des mécanismes de sécurité dans les processus DevOps. Ce travail met en évidence l'importance d'une approche DevSecOps adaptée aux environnements MLOps.

Mots-clés : MLOps, cybersécurité, CI/CD, vulnérabilités, CVE, NVD, DevSecOps.

Liste des abréviations

MLOps	Machine Learning Operations
CI/CD	Continuous Integration / Continuous Deployment
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
NVD	National Vulnerability Database
SAST	Static Application Security Testing
IaC	Infrastructure as Code
ML	Machine Learning
DevSecOps	Development, Security and Operations

Table des matières

Dédicace	ii
Remerciements	iii
Avant-propos	iv
Résumé	v
Liste des abréviations	vi
Liste des codes	ix
Liste des figures	x
Introduction	1
1 Première partie : Compte-rendu du stage	3
1.1 Définition du sujet du stage	3
1.1.1 Présentation générale du sujet	3
1.1.2 Contexte du projet	3
1.1.3 Importance du sujet	4
1.1.4 Objectifs du stage	4
1.2 Description du milieu de stage	5
1.2.1 Description de l'entreprise	5
1.2.2 Fonctionnement de l'entreprise	7
1.2.3 Superviseur de stage	7
1.3 Description du stage	8
1.3.1 Problématique	8
1.3.2 Environnement de travail	9
1.3.3 Responsabilités	9
1.3.4 Tâches	10
1.3.5 Déroulement chronologique du stage	10
1.3.6 Objectifs visés et résultats obtenus	11

2	Deuxième partie : Analyse et réflexion	12
2.1	Analyse du stage	12
2.1.1	Problèmes rencontrés	12
2.1.2	Solutions apportées	13
2.2	Impressions personnelles	13
2.2.1	Situations rencontrées lors du déroulement du stage	14
2.2.2	Apport du stage	15
2.2.3	Préparation du stage	16
3	Troisième partie : Annexes	18
3.1	Documents techniques	18
3.1.1	Description du workflow CI/CD sécurisé	18
3.1.2	Analyse multi-niveaux des vulnérabilités	19
3.1.3	Centralisation et enrichissement des vulnérabilités	19
3.1.4	Application d'une politique de sécurité	19
3.1.5	Déploiement sécurisé et validation	20
3.1.6	Synthèse technique	20
3.2	Schémas et diagrammes	20
3.2.1	Architecture globale	20
3.2.2	Pipeline CI/CD	21
3.2.3	Processus d'enrichissement des vulnérabilités	21
3.3	Extraits de code	22
3.3.1	Workflow CI/CD GitHub Actions	22
3.3.2	Script d'enrichissement des vulnérabilités	23
3.4	Documents complémentaires	28
3.4.1	Exécution du pipeline CI/CD	29
3.4.2	Détection des vulnérabilités	29
3.4.3	Rapport d'enrichissement des vulnérabilités	30
3.4.4	Validation de la politique de sécurité	33
3.4.5	Déploiement et validation	34
	Conclusion	35
	Bibliographie	36

Listings

3.1	Extrait du workflow GitHub Actions	22
3.2	Chargement et indexation de la base NVD	24
3.3	Traitement des vulnérabilités multi-sources	25
3.4	Mécanisme d'enrichissement	26
3.5	Génération du rapport consolidé	27
3.6	Apport du module d'enrichissement	28
3.7	Exemple de rapport d'enrichissement des vulnérabilités (CVE)	30

Table des figures

3.1	Architecture globale du système MLOps sécurisé	21
3.2	Pipeline CI/CD avec intégration DevSecOps	21
3.3	Enrichissement et corrélation des vulnérabilités avec la base NVD . .	22
3.4	Exécution complète du pipeline CI/CD	29
3.5	Résultats de l'analyse statique du code avec Bandit	29
3.6	Résultats de l'analyse des dépendances avec Trivy	30
3.7	Résultats de l'analyse des modèles avec ModelScan	30
3.8	Résultats de l'analyse des Dockerfiles avec Trivy	30
3.9	Blocage du pipeline en présence de vulnérabilités critiques	33
3.10	Validation du pipeline à l'absence de vulnérabilités critiques	33
3.11	Déploiement de l'application sur la machine virtuelle	34
3.12	Vérification du bon fonctionnement du service (Health Check)	34

Introduction

Dans un contexte où les systèmes basés sur l'intelligence artificielle occupent une place croissante dans les environnements de production, les pratiques de MLOps (Machine Learning Operations) se sont imposées comme une approche essentielle pour automatiser le cycle de vie des modèles d'apprentissage automatique. Ces pipelines permettent d'intégrer, de déployer et de maintenir des modèles de manière continue, en s'appuyant sur des outils issus du développement logiciel tels que l'intégration et le déploiement continu (CI/CD). Toutefois, cette automatisation introduit également de nouvelles surfaces d'attaque et expose les systèmes à des vulnérabilités souvent difficiles à détecter et à gérer.

Dans ce contexte, la gestion des vulnérabilités dans les pipelines MLOps constitue un enjeu majeur en cybersécurité. Les dépendances logicielles, les données d'entraînement, les modèles eux-mêmes ainsi que les infrastructures d'exécution peuvent être exploités par des acteurs malveillants, compromettant ainsi l'intégrité, la confidentialité et la disponibilité des systèmes. Malgré l'existence de certains outils de détection, leur intégration dans des environnements MLOps reste encore limitée et peu automatisée.

Le présent stage, réalisé à l'Université du Québec à Chicoutimi sous la supervision de Jonathan Roy, s'inscrit dans cette problématique. Il vise à concevoir et à mettre en œuvre une solution permettant d'automatiser la gestion des vulnérabilités dans les pipelines MLOps. L'objectif principal est de développer une approche capable d'identifier, d'analyser et de traiter les vulnérabilités de manière continue, en s'intégrant directement dans les processus existants.

Au-delà de l'aspect technique, ce projet présente également un intérêt pédagogique. En effet, la solution développée est destinée à être réutilisée dans le cadre de cours futurs, afin d'illustrer concrètement les enjeux de sécurité dans les environnements MLOps. Cette dimension confère au projet une portée supplémentaire, en contribuant à la formation des étudiants dans un domaine en pleine évolution.

Ce rapport présente, dans une première partie, le compte-rendu détaillé du stage, incluant la description du milieu, des activités réalisées et des résultats obtenus. La deuxième partie est consacrée à une analyse critique du stage, mettant en lumière les difficultés rencontrées, les solutions apportées ainsi que les apprentissages réali-

sés. Enfin, des annexes viennent compléter le document en apportant des éléments techniques supplémentaires.

Chapitre 1

Première partie : Compte-rendu du stage

1.1 Définition du sujet du stage

1.1.1 Présentation générale du sujet

Le stage porte sur l'automatisation de la gestion des vulnérabilités dans les pipelines MLOps, qui constituent un élément clé de la mise en production des modèles d'apprentissage automatique. Ces pipelines assurent l'enchaînement des différentes étapes nécessaires à leur exploitation, tout en garantissant la reproductibilité et la continuité des processus.

Cependant, leur complexité et leur forte dépendance à de multiples composants techniques rendent leur sécurisation particulièrement délicate. Les vulnérabilités peuvent apparaître à différents niveaux, notamment dans le code, les modèles, les bibliothèques utilisées, les configurations des conteneurs ou les infrastructures cloud. Une approche manuelle ne permet pas de répondre efficacement à ces enjeux, en raison du volume et de la fréquence des mises à jour.

Le projet consiste ainsi à développer une solution automatisée capable de détecter et de corréliser ces vulnérabilités, tout en intégrant des mécanismes de contrôle permettant de limiter la mise en production de composants à risque. Cette approche s'inscrit dans une démarche DevSecOps, visant à intégrer la sécurité de manière continue dans les processus de développement et de déploiement.

1.1.2 Contexte du projet

La généralisation de l'intelligence artificielle dans les environnements professionnels s'accompagne d'une augmentation des risques liés à la sécurité des systèmes. L'utilisation de pipelines automatisés facilite le déploiement rapide des modèles,

mais peut également introduire des failles si les mécanismes de contrôle ne sont pas adaptés.

Les menaces peuvent affecter aussi bien les modèles que leur environnement d'exécution, à travers des attaques ciblant les dépendances, les conteneurs ou les configurations. Dans ce contexte, la mise en place d'une gestion continue des vulnérabilités devient essentielle pour assurer la fiabilité et la sécurité des systèmes déployés.

Le stage s'inscrit dans cette dynamique et se déroule à l'Université du Québec à Chicoutimi, sous la supervision de M. Jonathan Roy. Il combine une dimension appliquée et pédagogique, la solution développée ayant vocation à être réutilisée dans un cadre académique afin d'illustrer les bonnes pratiques en matière de sécurité des environnements MLOps.

1.1.3 Importance du sujet

La gestion des vulnérabilités dans les pipelines MLOps représente un enjeu majeur pour plusieurs raisons. Premièrement, la présence de failles peut compromettre la confidentialité, l'intégrité et la disponibilité des systèmes. Une vulnérabilité dans un modèle ML ou dans son environnement d'exécution peut entraîner des décisions erronées ou des fuites de données sensibles.

Deuxièmement, les organisations doivent se conformer à des standards de sécurité et des exigences réglementaires. Un pipeline non sécurisé peut entraîner des risques juridiques, financiers et réputationnels importants.

Enfin, ce sujet est également pertinent sur le plan académique et pédagogique. La solution développée permet d'illustrer concrètement aux étudiants les enjeux de sécurité dans les environnements MLOps, leur donnant un exemple pratique de l'intégration du DevSecOps dans un cadre réel. Cette portée éducative renforce la valeur du projet au-delà de l'aspect purement technique.

1.1.4 Objectifs du stage

Le stage poursuit plusieurs objectifs complémentaires, à la fois techniques et pédagogiques :

Conception d'une architecture sécurisée pour les pipelines MLOps, avec une séparation claire des environnements de développement, d'intégration et de production, afin de limiter les risques liés aux interactions entre ces environnements. Automatisation de la détection et de la corrélation des vulnérabilités, couvrant le code applicatif, les dépendances, les modèles ML et les conteneurs. Mise en place de politiques de sécurité automatisées (Security Gates) pour bloquer toute mise en production d'artefacts présentant un risque critique. Traçabilité complète des analyses et des

décisions de sécurité, afin d’assurer la reproductibilité et la conformité des processus. Dimension pédagogique, en développant une solution réutilisable pour illustrer les bonnes pratiques de sécurité dans les cours futurs de MLOps et cybersécurité. Ces objectifs visent à produire une solution robuste, sécurisée et pédagogique, combinant des compétences techniques avancées avec une approche structurée applicable dans un contexte académique et professionnel.

1.2 Description du milieu de stage

Le stage s’est déroulé au sein de l’Université du Québec à Chicoutimi (UQAC), dans le cadre du département d’informatique et de mathématique, sous la supervision de M. Jonathan Roy, Ph.D.. Ce milieu de stage combine à la fois un environnement académique rigoureux et des infrastructures techniques avancées, permettant de réaliser un projet à la fois innovant, sécurisé et pédagogique. Le choix de ce laboratoire s’est avéré particulièrement pertinent pour un projet sur la gestion automatisée des vulnérabilités dans les pipelines MLOps, en raison de la convergence de compétences en cybersécurité, intelligence artificielle et pratiques DevSecOps.

1.2.1 Description de l’entreprise

Nom de l’organisation : Université du Québec à Chicoutimi

L’Université du Québec à Chicoutimi (UQAC) est une institution universitaire publique réputée pour son excellence dans l’enseignement et la recherche scientifique. Elle offre un large éventail de programmes académiques allant du premier cycle aux études doctorales, avec un fort accent sur l’innovation et la recherche appliquée. L’UQAC favorise l’émergence de projets concrets dans les domaines de la science, de l’ingénierie et des technologies numériques, permettant aux étudiants et chercheurs de travailler sur des problématiques directement applicables aux environnements industriels et académiques.

Secteur d’activité

L’UQAC opère principalement dans le secteur de l’éducation et de la recherche scientifique, en proposant des programmes d’études et des projets de recherche appliquée dans plusieurs disciplines. Le département d’informatique, où le stage s’est déroulé, se spécialise dans des domaines de pointe tels que la cybersécurité, l’intelligence artificielle et les systèmes distribués. Cette orientation permet de créer un environnement stimulant et propice à la réalisation de projets techniques ambitieux et novateurs, en lien direct avec les besoins émergents du marché de la cybersécurité

et des infrastructures MLOps.

Taille de l'organisation

L'UQAC accueille plusieurs milliers d'étudiants et un personnel académique et administratif conséquent. Le département d'informatique est composé d'une dizaine de chercheurs et enseignants-chercheurs, soutenus par des techniciens spécialisés et des assistants de recherche. Cette structure permet d'assurer un encadrement personnalisé tout en favorisant la collaboration multidisciplinaire. L'étudiant bénéficie ainsi d'un environnement où le soutien technique et scientifique est constant, tout en ayant la liberté nécessaire pour développer des solutions innovantes et tester des approches expérimentales.

Département d'accueil

Le stage s'est déroulé au sein du département d'informatique, plus particulièrement dans le laboratoire dirigé par M. Jonathan Roy. Ce laboratoire est spécialisé dans la cybersécurité appliquée aux systèmes intelligents, avec un accent particulier sur la sécurisation des pipelines MLOps et l'intégration de pratiques DevSecOps dans des environnements de développement automatisés. L'encadrement scientifique y est rigoureux, avec des objectifs de projet clairement définis et une disponibilité constante des ressources techniques et pédagogiques. Cet environnement permet non seulement de travailler sur des problématiques réelles, mais également d'expérimenter et de tester des solutions dans un cadre sécurisé et contrôlé.

Environnement matériel et logiciel

L'environnement technique du laboratoire combine ressources locales et cloud, offrant ainsi une infrastructure robuste et flexible pour le développement, le test et le déploiement des projets.

Machines locales : elles sont utilisées pour le développement du code applicatif, l'entraînement initial des modèles ML, les tests préliminaires et le versionnement via Git. Ces machines offrent un contrôle direct sur le code et les modèles, tout en garantissant une séparation claire entre le développement et les environnements de production. Infrastructure cloud (Microsoft Azure) : elle comprend des machines virtuelles configurées pour exécuter les pipelines CI/CD, les runners self-hosted GitHub Actions et le déploiement sécurisé des applications. L'utilisation du cloud permet de simuler des scénarios industriels, d'assurer une isolation stricte entre les environnements et de renforcer la sécurité du pipeline de production. Outils et logiciels : le laboratoire utilise Git et GitHub pour le versionnement et la gestion collaborative du code, Bandit, Trivy et ModelScan pour l'analyse des vulnérabilités, Docker pour

la conteneurisation, ainsi que des scripts personnalisés pour l’automatisation de la détection et de la gestion des vulnérabilités. L’ensemble de ces outils permet de reproduire un environnement MLOps réaliste, intégrant à la fois le développement logiciel et la sécurité des systèmes.

1.2.2 Fonctionnement de l’entreprise

Organisation du travail

Le laboratoire fonctionne selon un modèle collaboratif, structuré autour de projets définis et d’objectifs mesurables. Les étudiants travaillent soit individuellement, soit en petits groupes, sous la supervision directe du professeur. Chaque projet est planifié avec des jalons précis, ce qui permet de suivre l’avancement des travaux, d’identifier rapidement les problèmes et de proposer des solutions adaptées. Cette organisation favorise la discipline, la planification et le respect des bonnes pratiques scientifiques et techniques.

Mode de fonctionnement

Le laboratoire privilégie l’autonomie et la responsabilisation des étudiants, tout en garantissant un encadrement méthodologique constant. Les projets suivent un cycle de développement inspiré des bonnes pratiques DevSecOps : revue de code, tests automatisés, intégration continue et déploiement contrôlé. Les réunions régulières avec le superviseur permettent d’ajuster les choix techniques, de valider les résultats intermédiaires et de garantir la cohérence et la sécurité du projet. Ce mode de fonctionnement prépare les étudiants à évoluer dans des environnements professionnels exigeants, où rigueur et autonomie sont essentielles.

1.2.3 Superviseur de stage

Présentation du superviseur : Jonathan Roy

Le stage a été encadré par M. Jonathan Roy, Ph.D., professeur au département d’informatique et de mathématique de l’UQAC. M. Roy est un expert reconnu dans la cybersécurité appliquée à l’intelligence artificielle et dans la sécurisation des pipelines MLOps. Son expertise couvre plusieurs domaines :

Cybersécurité défensive, avec des méthodes avancées de prévention et de mitigation des vulnérabilités. Intelligence artificielle appliquée à la cybersécurité, incluant l’utilisation de modèles ML pour détecter et prévenir les menaces. Sécurité de l’intelligence artificielle, protégeant les modèles ML contre les manipulations et les attaques adversariales. Sécurité et qualité logicielle, en intégrant les bonnes pratiques

de développement sécurisé dans le cycle de vie logiciel. Sa supervision a permis d'assurer que le projet réponde à la fois à des exigences académiques élevées et à des standards industriels en matière de sécurité et d'automatisation.

Rôle dans l'entreprise

M. Roy a joué un rôle déterminant tout au long du stage, notamment :

Encadrement scientifique et technique : définition des objectifs, validation de l'architecture des pipelines MLOps et orientation sur les choix technologiques et méthodologiques. Orientation pédagogique : conception du projet pour qu'il soit réutilisable dans des cours futurs, permettant aux étudiants de comprendre concrètement les enjeux de sécurité dans les environnements MLOps. Suivi et mentorat : réunions régulières, conseils techniques et méthodologiques, validation des résultats et résolution des problèmes rencontrés. Cet encadrement a permis de concilier recherche appliquée, innovation technique et dimension pédagogique, offrant un cadre idéal pour développer des compétences avancées en cybersécurité et MLOps.

1.3 Description du stage

Cette section présente de manière détaillée le projet réalisé dans le cadre du stage, en mettant l'accent sur la problématique traitée, l'environnement de travail, les responsabilités assumées, les tâches effectuées, le déroulement chronologique ainsi que les objectifs visés et les résultats obtenus.

1.3.1 Problématique

Les pipelines MLOps modernes permettent d'automatiser l'ensemble du cycle de vie des modèles d'apprentissage automatique, depuis le développement jusqu'au déploiement en production. Cette automatisation apporte des gains importants en efficacité et en reproductibilité, mais elle introduit également une complexité accrue en matière de sécurité.

En effet, les systèmes MLOps reposent sur plusieurs couches interdépendantes : code applicatif, dépendances logicielles, modèles entraînés et infrastructures de déploiement. Chacune de ces couches peut contenir des vulnérabilités exploitables, ce qui augmente considérablement la surface d'attaque globale du système.

Dans la pratique, la gestion de ces vulnérabilités reste souvent fragmentée et peu intégrée aux pipelines automatisés. Les analyses de sécurité sont réalisées de manière ponctuelle, ce qui limite la capacité à détecter les risques en continu et à empêcher leur propagation vers les environnements de production.

La problématique centrale du stage consiste donc à répondre à la question suivante : comment concevoir un mécanisme automatisé, intégré aux pipelines MLOps, capable de détecter, corrélérer et contrôler les vulnérabilités de manière continue afin d’empêcher la mise en production de composants jugés à risque ?

1.3.2 Environnement de travail

Le stage a été réalisé dans un environnement hybride combinant développement local et infrastructure cloud, permettant de reproduire un pipeline MLOps réaliste et sécurisé.

L’environnement de travail repose sur une chaîne de développement structurée où le code est initialement développé sur une machine locale, puis versionné et envoyé vers un dépôt Git central. Chaque modification déclenche automatiquement un pipeline CI/CD exécuté sur une infrastructure distante.

Ce pipeline intègre plusieurs outils spécialisés permettant d’analyser différents aspects du système : le code source, les dépendances logicielles, les modèles d’apprentissage automatique ainsi que les conteneurs utilisés pour le déploiement. Les résultats de ces analyses sont ensuite consolidés afin de produire une vue unifiée des vulnérabilités détectées.

Une attention particulière a été portée à la séparation des environnements (développement, intégration et production), afin de garantir une isolation stricte et d’éviter toute propagation directe de vulnérabilités vers les systèmes critiques.

L’ensemble de cet environnement permet de simuler des conditions proches d’un contexte industriel, tout en conservant un cadre contrôlé adapté à la recherche et à l’expérimentation.

1.3.3 Responsabilités

Les responsabilités associées au stage consistaient principalement à participer à la conception et à la mise en œuvre d’un pipeline MLOps sécurisé intégrant des mécanismes automatisés de gestion des vulnérabilités.

Cela incluait la mise en place et la configuration des outils d’analyse de sécurité, ainsi que leur intégration dans un flux CI/CD automatisé. Une partie importante du travail consistait à assurer la cohérence entre les différentes étapes du pipeline afin de garantir une exécution fiable et reproductible.

Il était également nécessaire d’analyser et d’interpréter les résultats de détection de vulnérabilités afin de valider leur pertinence dans un contexte MLOps. Cette étape impliquait une réflexion critique sur les risques identifiés et leur impact potentiel sur le système global.

Enfin, le stage impliquait une responsabilité de documentation et de structuration du travail afin de rendre le système compréhensible, maintenable et réutilisable dans un cadre académique.

1.3.4 Tâches

Les tâches réalisées au cours du stage ont couvert l'ensemble du cycle de vie du pipeline de sécurité.

Une première phase a consisté en une étude des outils existants de détection de vulnérabilités afin de sélectionner ceux les plus adaptés aux besoins d'un environnement MLOps. Cette phase a permis de définir une stratégie d'intégration progressive des contrôles de sécurité.

Par la suite, les principaux outils de scan ont été intégrés dans le pipeline CI/CD afin d'automatiser l'analyse du code, des dépendances, des modèles et des conteneurs. Cette intégration a nécessité la configuration des workflows et leur adaptation aux contraintes du système.

Un autre volet important a concerné la mise en place d'un mécanisme de corrélation et de normalisation des vulnérabilités, permettant de regrouper les résultats issus de différentes sources et de les enrichir avec des informations de référence.

Le stage a également impliqué la définition de règles de sécurité permettant d'automatiser la prise de décision dans le pipeline, notamment le blocage du déploiement en présence de vulnérabilités critiques.

Enfin, des phases de tests et d'optimisation ont été réalisées afin d'améliorer la fiabilité du système, de réduire les faux positifs et de valider le bon fonctionnement global du pipeline dans différents scénarios.

1.3.5 Déroulement chronologique du stage

Le stage s'est déroulé de manière progressive, selon une approche structurée en plusieurs phases hebdomadaires permettant une montée en complexité du projet.

Les premières semaines ont été consacrées à une phase d'initiation et de recherche, visant à comprendre les concepts fondamentaux liés aux MLOps, à la cybersécurité et aux pratiques DevSecOps. Cette étape a permis de définir les bases du projet et d'identifier les outils pertinents.

Les semaines suivantes ont été dédiées à l'analyse progressive des différentes couches du système, incluant le code, les dépendances, les modèles et les conteneurs. Chaque couche a été étudiée séparément afin d'identifier les types de vulnérabilités associées.

Par la suite, une phase d'intégration dans le pipeline CI/CD a été réalisée, suivie de l'enrichissement des vulnérabilités et de la mise en place des mécanismes de

corrélation.

Les semaines intermédiaires ont été consacrées aux tests, à la validation du comportement du pipeline et à l'amélioration continue du système.

Enfin, les dernières semaines ont permis de finaliser le projet, de stabiliser le pipeline de déploiement continu, de réaliser les tests finaux et de préparer la documentation ainsi que les supports de présentation.

1.3.6 Objectifs visés et résultats obtenus

1.3.6 Objectifs visés et résultats obtenus

L'objectif principal du stage était de concevoir un pipeline MLOps sécurisé capable d'automatiser la détection, l'analyse et la gestion des vulnérabilités sur l'ensemble du cycle de vie des modèles d'apprentissage automatique.

Sur le plan technique, cet objectif a été atteint à travers la mise en place d'un système automatisé intégrant plusieurs outils de sécurité et appliquant des règles de contrôle permettant de bloquer les déploiements à risque. Le pipeline développé assure désormais une analyse continue et structurée des différentes composantes du système.

Sur le plan fonctionnel, la solution permet de centraliser les résultats de sécurité et de fournir une vision cohérente des vulnérabilités détectées, facilitant ainsi leur interprétation et leur gestion.

Enfin, sur le plan pédagogique, le projet a été conçu de manière à être réutilisable dans un contexte d'enseignement, permettant d'illustrer concrètement les enjeux de sécurité dans les environnements MLOps.

Globalement, les résultats obtenus répondent aux objectifs initiaux du stage en combinant automatisation, sécurité et reproductibilité, tout en offrant une base solide pour des extensions futures dans le domaine de la cybersécurité appliquée aux systèmes d'intelligence artificielle.

Chapitre 2

Deuxième partie : Analyse et réflexion

2.1 Analyse du stage

Cette section présente une analyse critique du déroulement du stage, en mettant en évidence les principaux problèmes rencontrés ainsi que les solutions mises en œuvre pour y remédier. Elle permet de prendre du recul sur les aspects techniques et organisationnels du projet.

2.1.1 Problèmes rencontrés

Au cours du stage, plusieurs difficultés techniques ont été rencontrées, principalement liées à l'intégration d'un pipeline de sécurité automatisé dans un contexte MLOps.

Un premier problème concernait l'hétérogénéité des résultats produits par les différents outils d'analyse de sécurité. Chaque outil utilisé dans le pipeline (analyse de code, dépendances, modèles ML et conteneurs) générait des sorties dans des formats distincts, ce qui compliquait leur agrégation et leur exploitation dans une vue unifiée des vulnérabilités.

Un second défi important était lié à la gestion des faux positifs et à l'interprétation des vulnérabilités dans un contexte MLOps. Certaines alertes générées par les outils de scan ne correspondaient pas nécessairement à des risques réels pour le système, ce qui nécessitait une analyse critique afin d'éviter une surévaluation du niveau de risque.

Une autre difficulté majeure concernait la mise en place de l'enrichissement des vulnérabilités à partir de la base de données NVD (National Vulnerability Database). L'association entre les vulnérabilités détectées et les entrées CVE officielles n'était pas toujours directe, en raison de différences de versions, d'informations incomplètes

ou de correspondances ambiguës. De plus, l'exploitation de la base NVD dans un environnement automatisé a posé des défis de performance et de structuration des données.

Enfin, la complexité de l'orchestration du pipeline CI/CD a constitué un obstacle supplémentaire. La coordination entre les différentes étapes de scan, d'enrichissement, de validation et de déploiement nécessitait une structuration rigoureuse afin de garantir la stabilité et la reproductibilité du système.

2.1.2 Solutions apportées

Pour répondre à ces différentes problématiques, plusieurs solutions techniques et méthodologiques ont été mises en œuvre.

Afin de traiter l'hétérogénéité des résultats, un mécanisme de normalisation et de centralisation des vulnérabilités a été développé. Celui-ci permet de transformer les sorties des différents outils en un format standardisé, facilitant leur agrégation et leur exploitation dans un rapport unique.

Pour la gestion des faux positifs, une analyse contextuelle des vulnérabilités dans un environnement MLOps a été adoptée. Les alertes ont été évaluées non seulement selon leur sévérité technique, mais également en fonction de leur impact réel sur le fonctionnement du pipeline et des modèles, permettant ainsi de mieux prioriser les risques.

Concernant l'enrichissement via la base NVD, une logique de correspondance flexible et tolérante aux incohérences a été mise en place. Elle combine recherche approximative, filtrage par dépendances et gestion des cas sans correspondance directe. Cette approche permet d'obtenir un enrichissement partiel mais exploitable, tout en conservant la traçabilité des vulnérabilités non identifiées.

Enfin, pour la gestion du pipeline CI/CD, une structuration stricte des étapes d'exécution a été implémentée, avec des dépendances explicites entre les jobs et des mécanismes de contrôle conditionnel. Cela garantit que chaque étape du pipeline ne s'exécute que si les conditions de sécurité sont respectées, assurant ainsi la cohérence globale du système.

2.2 Impressions personnelles

Cette section vise à présenter un retour critique et personnel sur l'expérience de stage. Elle met en lumière les situations vécues, les défis rencontrés, ainsi que les apprentissages réalisés sur les plans technique, humain et professionnel. Elle permet également d'évaluer l'apport global du stage dans le cadre de la formation en maîtrise en informatique, option cybersécurité.

2.2.1 Situations rencontrées lors du déroulement du stage

Relations humaines

Le stage s'est déroulé dans un environnement académique caractérisé par une interaction régulière avec le superviseur. Les échanges ont principalement pris la forme de réunions de suivi hebdomadaires, au cours desquelles les progrès réalisés, les difficultés rencontrées et les orientations futures étaient discutés.

Ces interactions ont permis de développer des compétences importantes en communication technique, notamment la capacité à expliquer des problématiques complexes de manière claire et structurée, ainsi qu'à défendre des choix techniques à l'aide d'arguments pertinents. Le cadre académique a encouragé une approche basée sur la réflexion, la justification et l'analyse critique, plutôt que sur une simple exécution de tâches.

Par ailleurs, bien que le travail ait été majoritairement individuel, l'environnement du laboratoire a favorisé un sentiment d'appartenance à une communauté scientifique. Cette dimension a contribué à renforcer la motivation et à maintenir un niveau d'exigence élevé tout au long du stage.

Adaptation

L'adaptation au stage a représenté un défi important, notamment en raison de la complexité du sujet et de la diversité des technologies impliquées. Le projet combinait des notions avancées de cybersécurité, de MLOps, de cloud computing et d'automatisation, nécessitant une montée en compétence rapide.

Au début du stage, une phase d'incertitude a été observée, liée à la difficulté de comprendre l'ensemble du système dans sa globalité. Toutefois, cette difficulté a progressivement été surmontée grâce à une approche méthodique, consistant à décomposer le problème en sous-parties et à traiter chaque composante de manière progressive.

L'adaptation a également concerné la gestion du travail de manière autonome. L'absence de structure rigide typique des environnements industriels a nécessité une organisation personnelle rigoureuse, incluant la planification des tâches, la gestion du temps et la priorisation des objectifs. Cette autonomie a constitué un apprentissage important dans le développement de compétences professionnelles.

Techniques

Sur le plan technique, le stage a exposé à des problématiques complexes rarement abordées de manière approfondie dans un cadre purement académique. L'intégration de multiples outils de sécurité dans un pipeline automatisé a nécessité une

compréhension fine de leur fonctionnement interne, ainsi que de leurs limites.

Un aspect particulièrement enrichissant a été la confrontation à des problèmes réels, tels que l'incohérence des résultats entre outils, les difficultés de corrélation des vulnérabilités ou encore les enjeux liés à l'automatisation des décisions de sécurité. Ces situations ont permis de développer une capacité à raisonner au-delà des outils, en adoptant une approche systémique du problème.

Le stage a également permis de renforcer les compétences en débogage, en analyse de logs et en résolution de problèmes complexes. La nécessité d'assurer la stabilité du pipeline a impliqué de nombreux ajustements techniques, contribuant ainsi à une meilleure maîtrise des environnements distribués et automatisés.

2.2.2 Apport du stage

Connaissances générales en informatique

Le stage a permis d'élargir significativement les connaissances générales en informatique, en particulier dans les domaines de la cybersécurité, des architectures distribuées et des pipelines d'intégration continue. Il a également permis de mieux comprendre les interactions entre différentes composantes d'un système complexe.

Cette expérience a mis en évidence l'importance d'une approche globale en informatique, où les aspects logiciels, systèmes et sécurité doivent être considérés conjointement pour garantir la fiabilité d'un système.

Niveau technique

Sur le plan technique, le stage a contribué à un développement significatif des compétences. Il a permis d'acquérir une expérience concrète dans la mise en œuvre de pipelines CI/CD sécurisés, ainsi que dans l'intégration d'outils d'analyse de vulnérabilités.

L'exposition à des technologies variées, notamment dans le domaine du cloud et des conteneurs, a également renforcé la capacité à travailler dans des environnements modernes et complexes. Le stage a permis de passer d'une compréhension théorique à une maîtrise pratique des concepts DevSecOps.

Connaissance du milieu de travail

Le stage a offert une immersion dans un environnement de recherche académique, mettant en évidence un mode de fonctionnement différent de celui des entreprises traditionnelles. L'accent est davantage mis sur l'exploration, l'expérimentation et la validation scientifique.

Cette expérience a permis de comprendre l'importance de la documentation, de la traçabilité des travaux et de la justification des choix techniques. Elle a également montré que la progression dans un projet de recherche est souvent non linéaire, nécessitant des ajustements continus.

Plan de carrière

Ce stage a eu un impact significatif sur la réflexion concernant le plan de carrière. Il a permis de confirmer un intérêt marqué pour les domaines de la cybersécurité appliquée aux systèmes intelligents et de l'automatisation des processus de sécurité.

Il a également mis en évidence l'importance des compétences interdisciplinaires, combinant sécurité, développement logiciel et intelligence artificielle. Cette expérience oriente vers des postes impliquant la conception de systèmes sécurisés dans des environnements complexes.

Enrichissement sur le plan personnel

Sur le plan personnel, le stage a contribué au développement de qualités telles que la persévérance, la rigueur et l'autonomie. La résolution de problèmes complexes a nécessité un effort continu et une capacité à faire face à l'incertitude.

Le stage a également permis de développer une meilleure gestion du stress et des délais, en particulier lors des phases critiques du projet. Cette expérience a renforcé la confiance en ses capacités à mener à bien des projets techniques de grande envergure.

Enrichissement comme élément de formation

Dans le cadre de la formation universitaire, le stage constitue une étape essentielle permettant de faire le lien entre théorie et pratique. Il a permis de consolider les connaissances acquises en cours tout en les appliquant dans un contexte réel.

Il a également mis en évidence certaines limites de la formation académique, notamment en ce qui concerne l'intégration de systèmes complexes et l'utilisation d'outils industriels. Cela souligne l'importance d'intégrer davantage de projets pratiques dans le cursus.

2.2.3 Préparation du stage

Niveau de connaissance général

Avant le stage, les connaissances acquises étaient principalement théoriques, avec une compréhension des concepts fondamentaux en informatique et en cybersécurité. Cependant, l'expérience pratique dans des environnements complexes restait limitée.

Le stage a permis de combler cet écart en offrant une immersion dans un projet concret, nécessitant l'application de connaissances dans un contexte réel.

Cours reliés

Plusieurs cours ont contribué à la réalisation du stage, notamment ceux liés à la cybersécurité, à l'intelligence artificielle, aux systèmes distribués et à l'ingénierie logicielle. Ces cours ont fourni les bases nécessaires pour comprendre les concepts abordés.

Cependant, le stage a montré que la combinaison de ces connaissances dans un même projet représente un défi supplémentaire, nécessitant une capacité d'intégration et d'adaptation.

Analyse critique de la formation universitaire

Le stage a permis de porter un regard critique sur la formation universitaire. Bien que celle-ci fournisse une base théorique solide, elle pourrait être enrichie par une exposition plus importante à des projets intégrateurs combinant plusieurs domaines.

Il serait également pertinent de renforcer les aspects pratiques liés aux outils utilisés en industrie, notamment dans les domaines du cloud, des pipelines CI/CD et de la cybersécurité appliquée.

Cette réflexion met en évidence l'importance de compléter la formation académique par des expériences pratiques telles que les stages, qui permettent de mieux préparer les étudiants aux réalités professionnelles.

Chapitre 3

Troisième partie : Annexes

Cette section regroupe l'ensemble des éléments techniques complémentaires permettant d'illustrer de manière détaillée les aspects les plus avancés du projet réalisé. Elle vise à fournir une compréhension approfondie de l'architecture, des mécanismes d'automatisation ainsi que des choix d'implémentation, sans alourdir le corps principal du rapport.

Les annexes présentées permettent notamment de mettre en évidence l'intégration des principes DevSecOps dans un contexte MLOps, ainsi que les solutions mises en œuvre pour automatiser la gestion des vulnérabilités.

3.1 Documents techniques

3.1.1 Description du workflow CI/CD sécurisé

Le projet repose sur un pipeline CI/CD automatisé implémenté à l'aide de GitHub Actions. Ce workflow constitue l'élément central du système, permettant d'orchestrer l'ensemble des opérations de détection, d'analyse, d'enrichissement et de gestion des vulnérabilités.

Le pipeline est déclenché automatiquement lors des opérations de *push* ou de *pull request* sur les branches principales du dépôt. Il est exécuté sur un environnement *self-hosted*, déployé sur une machine virtuelle, offrant ainsi un meilleur contrôle de l'infrastructure et permettant l'intégration de ressources locales, notamment la base de données NVD utilisée pour l'enrichissement des vulnérabilités.

L'exécution du workflow repose sur une organisation en plusieurs *jobs* dépendants, garantissant une progression séquentielle et contrôlée du processus de sécurité.

3.1.2 Analyse multi-niveaux des vulnérabilités

Le pipeline met en œuvre une approche multi-niveaux afin de couvrir les différentes surfaces d’attaque du système.

Dans un premier temps, une analyse statique du code source est effectuée afin d’identifier les vulnérabilités liées aux pratiques de développement. Cette étape permet de détecter des failles potentielles dès les premières phases du cycle de vie logiciel.

Ensuite, une analyse des dépendances est réalisée afin d’identifier les vulnérabilités connues associées aux bibliothèques utilisées. Cette étape est essentielle, car les dépendances externes constituent une source fréquente de vulnérabilités critiques.

Le pipeline intègre également une analyse spécifique des modèles d’apprentissage automatique, prenant en compte les risques propres aux environnements MLOps, notamment ceux liés à la désérialisation et à l’intégrité des modèles.

Enfin, une analyse des fichiers de configuration des conteneurs est effectuée afin de détecter les mauvaises pratiques de sécurité au niveau de l’infrastructure, telles que l’utilisation de configurations non sécurisées ou l’exposition inutile de services.

Cette approche globale permet d’assurer une couverture complète des risques, en intégrant à la fois les aspects logiciels, infrastructurels et spécifiques à l’intelligence artificielle.

3.1.3 Centralisation et enrichissement des vulnérabilités

Les résultats issus des différents outils d’analyse sont centralisés et traités par un module d’enrichissement développé en Python. Ce module permet de corréler les vulnérabilités détectées et de les enrichir à l’aide de données issues de la base officielle NVD.

L’utilisation d’une base locale présente plusieurs avantages, notamment une amélioration des performances, une réduction de la dépendance aux services externes et une meilleure maîtrise des données utilisées.

Le processus d’enrichissement permet d’associer des identifiants normalisés (CVE), des scores de sévérité (CVSS) ainsi que des informations complémentaires aux vulnérabilités détectées, facilitant ainsi leur analyse et leur exploitation.

3.1.4 Application d’une politique de sécurité

À l’issue de l’analyse et de l’enrichissement des vulnérabilités, une politique de sécurité automatisée est appliquée. Cette politique repose sur un mécanisme de *Security Gate*, permettant de contrôler le passage aux étapes suivantes du pipeline.

Le principe est de bloquer automatiquement le processus en présence de vulnérabilités critiques, quelle que soit leur origine (code, dépendances, modèles ou conteneurs). Cette approche permet d’éviter le déploiement de versions présentant un risque élevé.

Ce mécanisme s’inscrit dans une logique de *policy as code*, où les règles de sécurité sont directement intégrées dans le pipeline CI/CD.

3.1.5 Déploiement sécurisé et validation

Lorsque les conditions de sécurité sont satisfaites, le pipeline déclenche automatiquement le processus de déploiement sur une machine distante.

Ce processus comprend la récupération du code source, la construction de l’image Docker, ainsi qu’une analyse de sécurité de l’image générée afin de détecter d’éventuelles vulnérabilités introduites lors du *build*.

Une fois le conteneur déployé, une vérification post-déploiement est effectuée à travers un test de santé basé sur un appel HTTP. Cette étape permet de valider le bon fonctionnement du service et de s’assurer de la disponibilité de l’application.

3.1.6 Synthèse technique

Le workflow implémenté présente plusieurs caractéristiques majeures :

- Une automatisation complète du processus de sécurité
- Une approche multi-couches de détection des vulnérabilités
- Une centralisation et un enrichissement des résultats
- Une intégration de politiques de sécurité dans le pipeline
- Un déploiement conditionnel basé sur le niveau de risque

Ce système constitue une implémentation concrète des principes DevSecOps appliqués aux environnements MLOps.

3.2 Schémas et diagrammes

Cette sous-section regroupe les représentations graphiques permettant de visualiser l’architecture globale du système ainsi que le fonctionnement du pipeline mis en place.

3.2.1 Architecture globale

Un premier schéma présente l’architecture générale du système. Il met en évidence la séparation entre les environnements de développement, d’intégration conti-

nue et de production. Cette organisation permet de renforcer l'isolation des composants et de limiter les risques de compromission.

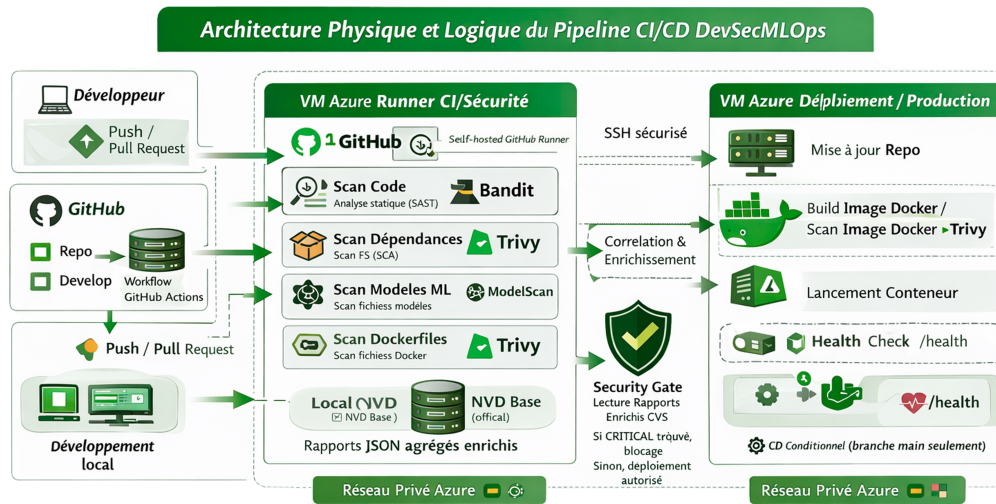


FIGURE 3.1 – Architecture globale du système MLOps sécurisé

3.2.2 Pipeline CI/CD

Un second schéma illustre le déroulement du pipeline CI/CD. Il met en évidence l'enchaînement des différentes étapes d'analyse de sécurité ainsi que les dépendances entre les différents modules. Ce schéma permet de mieux comprendre la logique de fonctionnement du système et le rôle de chaque étape dans le processus global.

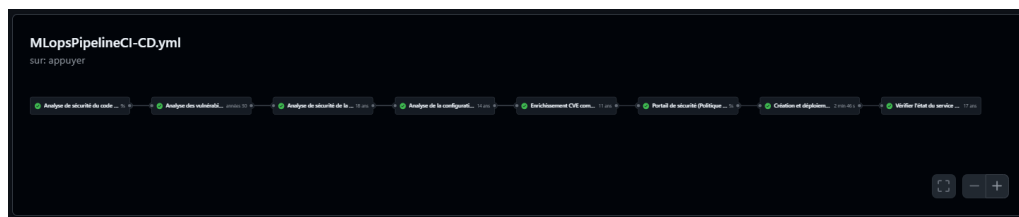


FIGURE 3.2 – Pipeline CI/CD avec intégration DevSecOps

3.2.3 Processus d'enrichissement des vulnérabilités

Un troisième schéma est consacré au mécanisme d'enrichissement des vulnérabilités. Il montre comment les résultats issus de différents outils d'analyse sont centralisés, puis corrélés avec les données de la base NVD afin de produire un rapport unifié. Ce processus permet d'améliorer la qualité de l'analyse et d'obtenir une vision consolidée des risques.

Processus d'enrichissement des vulnérabilités

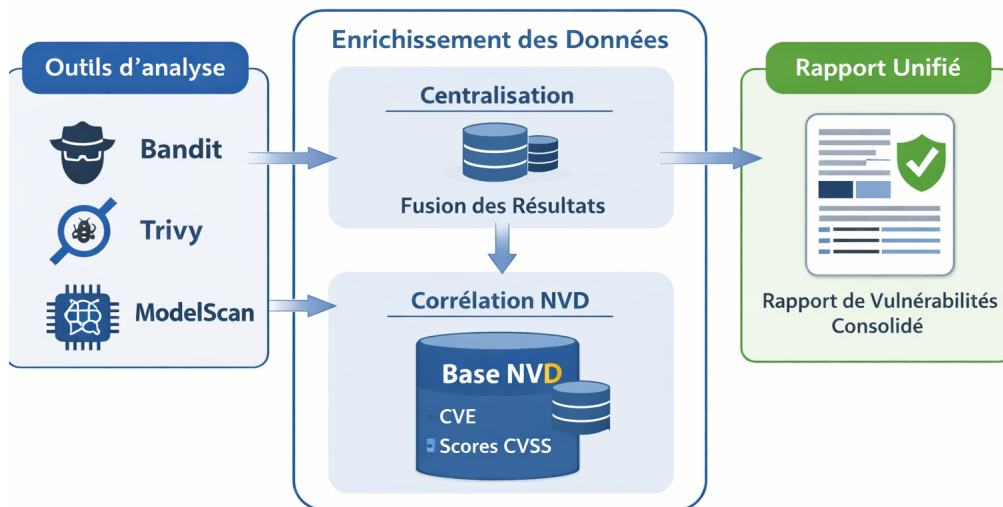


FIGURE 3.3 – Enrichissement et corrélation des vulnérabilités avec la base NVD

3.3 Extraits de code

Cette section présente les principaux extraits de code développés dans le cadre du projet. Elle met en évidence, d'une part, le workflow d'automatisation implémenté avec GitHub Actions, et d'autre part, le script d'enrichissement des vulnérabilités basé sur l'exploitation de la base de données NVD.

3.3.1 Workflow CI/CD GitHub Actions

Le workflow constitue le cœur du système DevSecOps mis en place. Il permet d'orchestrer l'ensemble des étapes d'analyse de sécurité, d'enrichissement des vulnérabilités et de validation avant le déploiement.

Le pipeline est déclenché automatiquement lors d'un push ou d'une pull request sur les branches principales du projet. Il est exécuté sur une infrastructure self-hosted, garantissant un meilleur contrôle des ressources et permettant l'utilisation d'une base NVD locale.

Listing 3.1 – Extrait du workflow GitHub Actions

```
1 name: Full Project Automatisation de la gestion des
   vuln rabil it s
2
3 on:
4   push:
5     branches:
6       - main
```

```

7     - Develop
8 pull_request:
9     branches:
10    - main
11    - Develop
12
13 jobs:
14   CodeScan:
15     runs-on: [self-hosted, Linux, X64]
16     steps:
17       - uses: actions/checkout@v4
18       - name: Run Bandit
19         run: |
20           bandit -r . -f json -o bandit-report.json || true
21
22   DependencyScan:
23     runs-on: [self-hosted, Linux, X64]
24     needs: CodeScan
25     steps:
26       - uses: actions/checkout@v4
27       - name: Run Trivy
28         run: |
29           trivy fs --format json --output trivy-report.json .
30
31   CVEEnrichment:
32     runs-on: [self-hosted, Linux, X64]
33     needs: [CodeScan, DependencyScan]
34     steps:
35       - uses: actions/checkout@v4
36       - name: Run enrichment script
37         run: |
38           python scripts/cve_enrichment.py \
39             --nvd-db /nvd_data \
40             --bandit-report bandit-report.json \
41             --trivy-report trivy-report.json \
42             --output enriched.json

```

Cet extrait illustre l'enchaînement logique des étapes du pipeline, ainsi que l'intégration du module d'enrichissement dans le processus global.

3.3.2 Script d'enrichissement des vulnérabilités

Le script d'enrichissement constitue un élément clé du système développé. Il permet de centraliser les résultats issus des différents outils d'analyse et de les enrichir à l'aide des données de la base NVD.

L'objectif principal de ce script est de transformer des résultats hétérogènes en

une structure unifiée, facilitant l'analyse et la prise de décision.

Chargement et indexation de la base NVD

Le script débute par le chargement des données issues de la base de données NVD (National Vulnerability Database), stockée localement sur l'infrastructure d'exécution. Cette base contient des informations détaillées sur les vulnérabilités connues, notamment les identifiants CVE, les scores CVSS et les classifications CWE.

Afin d'optimiser les performances du processus d'enrichissement, une phase d'indexation est réalisée en amont. Cette indexation repose sur les identifiants CWE (Common Weakness Enumeration), permettant d'établir une correspondance entre les faiblesses détectées dans le code et les vulnérabilités référencées dans la NVD.

Listing 3.2 – Chargement et indexation de la base NVD

```
1 nvd_cwe_index = {}
2 for nvd_file in glob.glob(os.path.join(args.nvd_db, "*.json")):
3     with open(nvd_file, "r", encoding="utf-8") as f:
4         data = json.load(f)
5
6     for vuln in data.get("vulnerabilities", []):
7         cve = vuln.get("cve", {})
8         for w in cve.get("weaknesses", []):
9             for desc in w.get("description", []):
10                 if desc.get("value", "").startswith("CWE-"):
11                     cwe = desc["value"].replace("CWE-", "")
12                     nvd_cwe_index.setdefault(cwe, []).append(vuln)
```

Cette approche permet d'éviter des recherches coûteuses lors du traitement et d'assurer une corrélation rapide et efficace entre les différentes sources d'information. Le script commence par charger les données issues de la base NVD stockée localement. Une indexation est réalisée en utilisant les identifiants CWE.

Traitement des vulnérabilités multi-sources

Le script est conçu pour traiter simultanément plusieurs types de vulnérabilités, provenant de différentes sources d'analyse. Cette capacité multi-sources constitue un élément essentiel dans un contexte MLOps, où les surfaces d'attaque sont variées.

Les principales catégories traitées sont les suivantes :

- Vulnérabilités du code issues de l'analyse statique (SAST), permettant d'identifier des failles liées aux pratiques de développement ;
- Vulnérabilités des dépendances, associées aux bibliothèques externes utilisées par l'application, généralement identifiées par des CVE connus ;

-
- Vulnérabilités des modèles d'apprentissage automatique, spécifiques aux environnements MLOps, notamment liées à la désérialisation ou à l'intégrité des modèles ;
 - Vulnérabilités des conteneurs, résultant de mauvaises configurations dans les fichiers Docker ou dans l'infrastructure de déploiement.

Listing 3.3 – Traitement des vulnérabilités multi-sources

```
1 # =====
2 # Charger Trivy (DEPENDANCES)
3 # =====
4 dependency_vulns = []
5 for result in trivy_data.get("Results", []):
6     for vuln in result.get("Vulnerabilities", []) or []:
7         dependency_vulns.append({
8             "type": "dependency",
9             "package": vuln.get("PkgName"),
10            "version": vuln.get("InstalledVersion"),
11            "cve": vuln.get("VulnerabilityID"),
12            "severity": vuln.get("Severity"),
13            "cvss": vuln.get("CVSS", {}),
14            "description": vuln.get("Description") or vuln.get("
15                Title", "")
16        })
17 # =====
18 # Charger ModelScan (MODELS)
19 # =====
20 model_vulns = [
21     {
22         "type": "model",
23         "file": v.get("file"),
24         "description": v.get("description"),
25         "severity": v.get("severity"),
26         "tool": v.get("tool", "ModelScan"),
27         "cve": v.get("cve"),
28         "id": v.get("id")
29     }
30     for v in modelscan_data.get("vulnerabilities", [])
31     if v.get("severity") != "UNKNOWN"
32 ]
33
34 # =====
35 # Charger ContainerScan (Dockerfiles)
36 # =====
37 container_vulns = []
38
```

```

39 if args.container_reports and os.path.exists(args.container_reports
):
40     container_files = glob.glob(os.path.join(args.container_reports
, "**/*.json"), recursive=True)
41
42     for f in container_files:
43         with open(f, "r") as jfile:
44             data = json.load(jfile)
45             for result in data.get("Results", []):
46                 target = result.get("Target")
47                 for vuln in result.get("Misconfigurations", []) or
[]:
48                     container_vulns.append({
49                         "type": "container",
50                         "dockerfile": target,
51                         "vulnerability_id": vuln.get("ID"),
52                         "severity": vuln.get("Severity"),
53                         "rule": vuln.get("RuleID"),
54                         "description": vuln.get("Message"),
55                         "tool": "Trivy"
56                     })

```

Chaque catégorie est extraite, normalisée et intégrée dans une structure de données homogène, facilitant ainsi les étapes ultérieures de traitement et d'analyse.

Mécanisme d'enrichissement

Le processus d'enrichissement constitue le cœur fonctionnel du script. Il repose sur la correspondance entre les identifiants CWE détectés dans les vulnérabilités du code et les entrées correspondantes dans la base NVD.

Lorsque cette correspondance est établie, le script enrichit automatiquement les vulnérabilités avec des informations complémentaires, notamment :

- des identifiants CVE permettant une référence standardisée ;
- des scores CVSS (Common Vulnerability Scoring System), offrant une mesure quantitative du niveau de risque ;
- des niveaux de sévérité normalisés (LOW, MEDIUM, HIGH, CRITICAL), facilitant la prise de décision.

Listing 3.4 – Mécanisme d'enrichissement

```

1 if cwe in nvd_cwe_index:
2     cve_item = nvd_cwe_index[cwe][0]["cve"]
3
4     metrics = cve_item.get("metrics", {}).get("cvssMetricV31", [])
5     if metrics:
6         cvss = metrics[0]["cvssData"]

```



```

7
8     enriched.update({
9         "mapped_cve": cve_item.get("id"),
10        "cvss_score": cvss.get("baseScore"),
11        "cvss_vector": cvss.get("vectorString"),
12        "nvd_severity": cvss.get("baseSeverity")
13    })

```

Ce mécanisme permet d’uniformiser les résultats issus de différents outils, tout en augmentant leur pertinence et leur exploitabilité dans un contexte de gestion automatisée des risques.

Génération du rapport consolidé

À l’issue du traitement, le script génère un rapport final structuré au format JSON. Ce rapport regroupe l’ensemble des vulnérabilités identifiées et enrichies, en les classant par catégorie.

Il contient notamment :

- un résumé global indiquant le nombre de vulnérabilités par type ;
- une liste détaillée des vulnérabilités pour chaque catégorie (code, dépendances, modèles, conteneurs) ;
- des informations enrichies facilitant l’analyse et la priorisation des risques.

Listing 3.5 – Génération du rapport consolidé

```

1 final_report = {
2     "summary": {
3         "code_issues": len(code_vulns),
4         "dependency_issues": len(dependency_vulns),
5         "model_issues": len(model_vulns),
6         "container_issues": len(container_vulns)
7     },
8     "code": code_vulns,
9     "dependencies": dependency_vulns,
10    "models": model_vulns,
11    "containers": container_vulns
12 }
13
14 with open(args.output, "w") as f:
15     json.dump(final_report, f, indent=2)

```

Ce rapport constitue une source unique de vérité pour le pipeline, utilisée notamment par le module de contrôle (Security Gate) afin de déterminer si le déploiement peut être autorisé.

Apport du module d'enrichissement

Le module d'enrichissement apporte une valeur ajoutée significative au système global. Il permet non seulement d'améliorer la qualité des données de sécurité, mais également de renforcer la cohérence et l'intégration des différents composants du pipeline.

En centralisant et en normalisant les informations issues de sources multiples, il facilite leur interprétation et leur exploitation dans un contexte automatisé. De plus, en intégrant des données issues de la NVD, il permet d'ancrer l'analyse dans des référentiels reconnus, renforçant ainsi la fiabilité des décisions prises.

Listing 3.6 – Apport du module d'enrichissement

```
1 - name: Security Gate
2   run: |
3     python3 - << EOF
4     import json, sys
5
6     with open("enriched_cves.json") as f:
7         data = json.load(f)
8
9     critical = []
10
11    for item in data.get("dependencies", []):
12        if item.get("severity") == "CRITICAL":
13            critical.append(item["cve"])
14
15    if critical:
16        print("CRITICAL vulnerabilities detected")
17        sys.exit(1)
18
19    print("No critical vulnerabilities")
20    EOF
```

Ce module joue ainsi un rôle central dans la transformation du pipeline en un système intelligent de gestion des vulnérabilités, capable de supporter des mécanismes avancés de contrôle et de validation avant déploiement.

3.4 Documents complémentaires

Cette section regroupe l'ensemble des éléments supplémentaires permettant d'illustrer concrètement le fonctionnement du système développé. Elle vise à appuyer les résultats présentés dans le rapport en fournissant des preuves visuelles et techniques du bon déroulement du pipeline ainsi que de l'efficacité des mécanismes de sécurité mis en œuvre.

3.4.1 Exécution du pipeline CI/CD

Cette figure illustre l'exécution complète du pipeline CI/CD, incluant les différentes étapes d'analyse de sécurité, d'enrichissement des vulnérabilités et de déploiement.

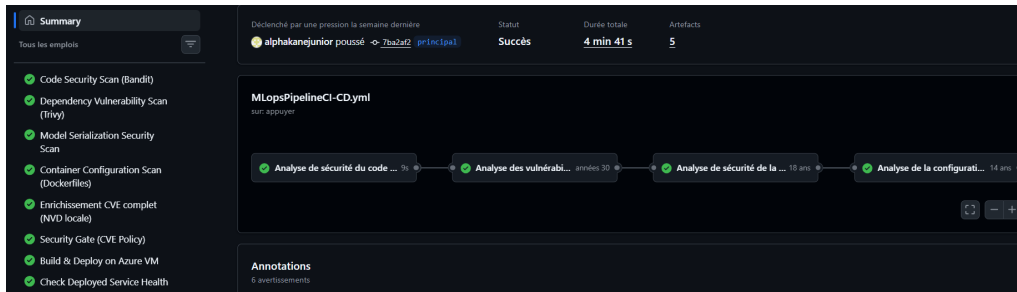


FIGURE 3.4 – Exécution complète du pipeline CI/CD

On observe que les différents jobs sont exécutés de manière séquentielle, garantissant une analyse progressive et contrôlée du système.

3.4.2 Détection des vulnérabilités

Les outils d'analyse intégrés dans le pipeline permettent de détecter plusieurs types de vulnérabilités, notamment au niveau du code source, des dépendances, les modèles et les fichiers de configurations docker .

```
29 ***** Rapport_code_vulns *****
30
31 📁 CODE VULNERABILITIES
32 | File | Line | CVE | Severity | Issue |
33 |-----|-----|-----|-----|-----|
34 | ./ModelApp/app.py | 45 | | | Possible binding to all interfaces. |
35 | ./Vulnerability/unsafe-code.py | 3 | | | Consider possible security implications associated with pickle module. |
36 | ./Vulnerability/unsafe-code.py | 4 | | | Consider possible security implications associated with the subprocess module. |
37 | ./Vulnerability/unsafe-code.py | 13 | | | Use of unsafe yaml load. Allows instantiation of arbitrary objects. Consider yaml.safe_load(). |
38 | ./Vulnerability/unsafe-code.py | 22 | | | Starting a process with a shell, possible injection detected, security issue. |
39 | ./Vulnerability/unsafe-code.py | 31 | | | Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue. |
40 | ./Vulnerability/unsafe-code.py | 40 | | | subprocess call with shell=True identified, security issue. |
41 | ./Vulnerability/unsafe-code.py | 50 | | | tarfile.extractall used without any validation. Please check and discard dangerous members. |
```

FIGURE 3.5 – Résultats de l'analyse statique du code avec Bandit

```

45 ***** Rapport_dependency_vulns *****
46
47 🚩 DEPENDENCY VULNERABILITIES
48 | Package | Version | CVE | Severity | Description
49 |-----|-----|-----|-----|-----|
50 | Flask | 3.1.2 | CVE-2026-27285 | LOW | Flask is a web server gateway interface (WSGI) web application framework. In versions 3.1.2 and below, when
51 | requests | 2.32.3 | CVE-2024-47881 | MEDIUM | Requests is a HTTP library. Due to a URL parsing issue, Requests releases prior to 2.32.4 may leak .netrc
52 | requests | 2.32.3 | CVE-2026-25645 | MEDIUM | Requests is a HTTP library. Prior to version 2.33.0, the 'requests.utils.extract_zipped_paths()' utility
53 | scikit-learn | 1.3.2 | CVE-2024-5286 | MEDIUM | A sensitive data leakage vulnerability was identified in scikit-learn's TfidfVectorizer, specifically in
  versions up to |

```

FIGURE 3.6 – Résultats de l’analyse des dépendances avec Trivy

```

55 ***** Rapport_model_vulns *****
56
57 🚩 MODEL VULNERABILITIES
58 | File | Description | Severity | Tool |
59 |-----|-----|-----|-----|
60 | /home/azureuser/actions-runner/_work/MLOps-Automation-Vulnerability-Detection/MLOps-Automation-Vulnerability-
61 | Detection/ModelApp/model/unsafe_model.pt:unsafe_zip_pytorch/data.pkl | Use of unsafe operator 'system' from module 'posix' | CRITICAL | ModelScan |

```

FIGURE 3.7 – Résultats de l’analyse des modèles avec ModelScan

```

62 ***** Rapport_dockerfile_vulns *****
63
64 🚩 DOCKERFILE VULNERABILITIES
65 | Dockerfile | VulnID | Severity | Rule | Description |
66 |-----|-----|-----|-----|-----|
67 | Dockerfile | DS-0002 | HIGH | | Specify at least 1 USER command in Dockerfile with non-root user as argument |
68 | Dockerfile | DS-0026 | LOW | | Add HEALTHCHECK instruction in your Dockerfile |
69
70 ✅ Rapport enrichi généré : reports/cve_enrichment/23973482691/enriched_cves.json

```

FIGURE 3.8 – Résultats de l’analyse des Dockerfiles avec Trivy

Les résultats mettent en évidence des vulnérabilités classées selon leur niveau de sévérité, facilitant leur identification et leur priorisation.

3.4.3 Rapport d’enrichissement des vulnérabilités

Le module d’enrichissement génère un rapport consolidé au format JSON, intégrant les vulnérabilités issues de différentes sources.

Listing 3.7 – Exemple de rapport d’enrichissement des vulnérabilités (CVE)

```

1 {
2   "summary": {
3     "code_issues": 2,
4     "dependency_issues": 1,
5     "model_issues": 1,
6     "container_issues": 2
7   },
8   "code": [
9     {
10      "type": "code",

```

```

11     "file": "./ModelApp/app.py",
12     "line": 45,
13     "cwe": "605",
14     "issue": "Possible binding to all interfaces.",
15     "severity": "MEDIUM",
16     "confidence": "MEDIUM",
17     "test_id": "B104",
18     "source": "Bandit"
19 },
20 {
21     "type": "code",
22     "file": "./Vulnerability/unsafe-code.py",
23     "line": 3,
24     "cwe": "502",
25     "issue": "Consider possible security implications associated
26             with pickle module.",
27     "severity": "LOW",
28     "confidence": "HIGH",
29     "test_id": "B403",
30     "source": "Bandit"
31 },
32 "dependencies": [
33     {
34         "type": "dependency",
35         "package": "Flask",
36         "version": "3.1.2",
37         "cve": "CVE-2026-27205",
38         "severity": "LOW",
39         "cvss": {
40             "ghsa": {
41                 "V40Vector": "CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:P/VC:L/VI:N
42                             /VA:N/SC:N/SI:N/SA:N",
43                 "V40Score": 2.3
44             },
45             "nvd": {
46                 "V3Vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N
47                             ",
48                 "V3Score": 4.3
49             },
50             "redhat": {
51                 "V3Vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N
52                             ",
53                 "V3Score": 4.3
54             }
55         }
56     },
57     "description": "Flask is a web server gateway interface (WSGI

```

```

    ) web application framework. In versions 3.1.2 and below,
    when the session object is accessed, Flask should set the
    Vary: Cookie header., resulting in a Use of Cache
    Containing Sensitive Information vulnerability. The logic
    instructs caches not to cache the response, as it may
    contain information specific to a logged in user. This is
    handled in most cases, but some forms of access such as
    the Python in operator were overlooked. The severity and
    risk depend on the application being hosted behind a
    caching proxy that doesn't ignore responses with cookies,
    not setting a Cache-Control header to mark pages as
    private or non-cacheable, and accessing the session in a
    way that only touches keys without reading values or
    mutating the session. The issue has been fixed in version
    3.1.3."
54     }
55 ],
56 "models": [
57     {
58         "type": "model",
59         "file": "/home/azureuser/actions-runner/_work/Mlops -
            Automation-Vulrability-Detection/Mlops-Automation -
            Vulrability-Detection/ModelApp/model/unsafe_model.pt:
            unsafe_zip_pytorch/data.pkl",
60         "description": "Use of unsafe operator 'system' from module '
            posix'",
61         "severity": "CRITICAL",
62         "tool": "ModelScan",
63         "cve": "modelscan-6431835936303175400",
64         "id": "modelscan-6431835936303175400"
65     }
66 ],
67 "containers": [
68     {
69         "type": "container",
70         "dockerfile": "Dockerfile",
71         "vulnerability_id": "DS-0002",
72         "severity": "HIGH",
73         "rule": null,
74         "description": "Specify at least 1 USER command in Dockerfile
            with non-root user as argument",
75         "tool": "Trivy"
76     },
77     {
78         "type": "container",
79         "dockerfile": "Dockerfile",
80         "vulnerability_id": "DS-0026",

```

```

81     "severity": "LOW",
82     "rule": null,
83     "description": "Add HEALTHCHECK instruction in your
84         Dockerfile",
85     "tool": "Trivy"
86 }
87 ]

```

Ce rapport associe des identifiants CVE, des scores CVSS ainsi que des niveaux de sévérité normalisés, facilitant ainsi l'analyse globale des risques.

3.4.4 Validation de la politique de sécurité

Le pipeline intègre un mécanisme de contrôle permettant de bloquer le déploiement en cas de vulnérabilités critiques.

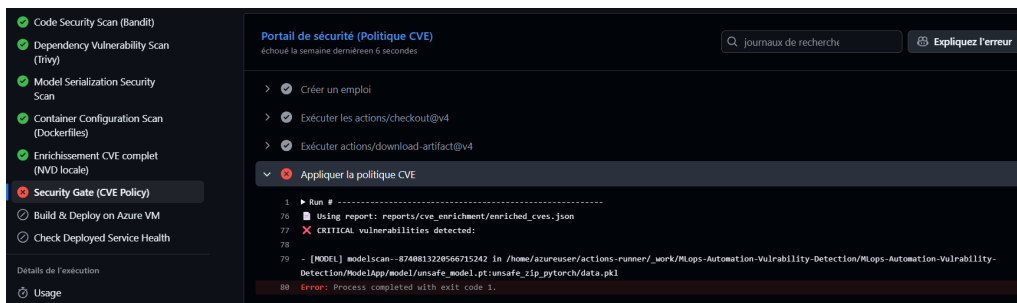


FIGURE 3.9 – Blocage du pipeline en présence de vulnérabilités critiques



FIGURE 3.10 – Validation du pipeline à l'absence de vulnérabilités critiques

Cette étape garantit que seules les versions respectant les exigences de sécurité peuvent être déployées.

3.4.5 Déploiement et validation

Une fois les conditions de sécurité validées, le pipeline déclenche automatiquement le déploiement de l'application sur une machine distante.

```
473 Legend:
474 - '-': Not scanned
475 - '0': Clean (no security findings detected)
476 ✅ Aucune vulnérabilité CRITICAL détectée
477 🛑 Arrêt ancien conteneur
478 Error response from daemon: No such container: mlapp
479 Error response from daemon: No such container: mlapp
480 🚀 Démarrage nouveau conteneur
481 acf4d3a7d2c60ed69b1f15e3e2f0e3ff04bb33a2056ccdd7292c0c6cbb608a8a
482 🚀 Déploiement sécurisé terminé
483 =====
484 ✅ Successfully executed commands to all hosts.
485 =====

> 🟢 Travail terminé
```

FIGURE 3.11 – Déploiement de l'application sur la machine virtuelle

Un test de santé est ensuite effectué afin de vérifier la disponibilité du service.

```

✅ Code Security Scan (Bandit)
✅ Dependency Vulnerability Scan (Trivy)
✅ Model Serialization Security Scan
✅ Container Configuration Scan (Dockerfiles)
✅ Enrichissement CVE complet (NVD locale)
✅ Security Gate (CVE Policy)
✅ Build & Deploy on Azure VM
✅ Check Deployed Service Health

Détails de l'exécution
🔄 Usage
📄 Workflow file

> 🟢 Créer un emploi
> 🟢 Attendez que le conteneur démarre
  📄 bilan de santé
1 ▶ Run curl -v http://$PUBLIC_IP:80/health || echo "Server unreachable"
9 % Total % Received % Xferd Average Speed Time Time Time Current
10 % Dload Upload Total Spent Left Speed
11
12 0 0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0* Trying ***:80...
13 * Connected to *** (***) port 80
14 > GET / HTTP/1.1
15 > Host: ***
16 > User-Agent: curl/8.5.0
17 > Accept: */*
18 >
19 < HTTP/1.1 200 OK
20 < Server: Werkzeug/3.1.8 Python/3.12.13
21 < Date: Sat, 04 Apr 2026 19:34:04 GMT
22 < Content-Type: text/html; charset=utf-8
23 < Content-Length: 1787
24 < Connection: close
```

FIGURE 3.12 – Vérification du bon fonctionnement du service (Health Check)

Les résultats confirment le bon fonctionnement de l'application en environnement cible.

Conclusion

Ce stage a permis de concevoir et de mettre en œuvre une solution complète visant à renforcer la sécurité des pipelines MLOps par l'automatisation de la gestion des vulnérabilités. Dans un contexte où les systèmes d'intelligence artificielle sont de plus en plus déployés en production, la maîtrise des risques de sécurité associés à leur cycle de vie devient essentielle.

L'approche développée repose sur l'intégration d'un pipeline CI/CD sécurisé capable de détecter, d'analyser et de traiter les vulnérabilités de manière continue. La solution proposée se distingue par une analyse multi-niveaux couvrant le code source, les dépendances, les modèles d'apprentissage automatique ainsi que les configurations des conteneurs. Cette couverture étendue permet de prendre en compte les différentes surfaces d'attaque propres aux environnements MLOps.

L'un des apports majeurs du projet réside dans le développement d'un module d'enrichissement basé sur la base de données NVD. Ce module permet d'associer aux vulnérabilités détectées des informations normalisées telles que les identifiants CVE et les scores CVSS, facilitant ainsi leur analyse et leur priorisation. Par ailleurs, l'intégration d'un mécanisme de Security Gate assure un contrôle automatisé du pipeline en bloquant le déploiement en présence de vulnérabilités critiques.

Les résultats obtenus démontrent l'efficacité de la solution, tant en termes d'automatisation que d'amélioration de la sécurité globale. Le système développé permet une détection précoce des vulnérabilités et une prise de décision basée sur des critères objectifs, contribuant ainsi à renforcer la fiabilité des déploiements.

Ce stage a également permis de consolider des compétences avancées en cybersécurité, en intégration continue et en gestion des systèmes MLOps, tout en offrant une expérience concrète de mise en œuvre dans un environnement réel.

Enfin, des perspectives d'amélioration peuvent être envisagées, notamment l'intégration d'analyses dynamiques, l'automatisation avancée des réponses aux incidents ou encore l'adaptation de la solution à des environnements distribués.

En conclusion, ce projet constitue une contribution pertinente à la sécurisation des pipelines MLOps et s'inscrit dans une démarche d'évolution vers des systèmes plus robustes, automatisés et sécurisés.

Bibliographie

- [1] OpenStack Foundation, *Bandit Documentation*, Disponible sur : <https://bandit.readthedocs.io>, consulté en avril 2026.
- [2] Aqua Security, *Trivy Documentation*, Disponible sur : <https://trivy.dev>, consulté en avril 2026.
- [3] Protect AI, *ModelScan Documentation*, Disponible sur : <https://github.com/protectai/modelscan>, consulté en avril 2026.
- [4] National Institute of Standards and Technology (NIST), *National Vulnerability Database (NVD)*, Disponible sur : <https://nvd.nist.gov>, consulté en avril 2026.
- [5] MITRE Corporation, *Common Vulnerabilities and Exposures (CVE)*, Disponible sur : <https://cve.mitre.org>, consulté en avril 2026.
- [6] GitHub, *GitHub Actions Documentation*, Disponible sur : <https://docs.github.com/en/actions>, consulté en avril 2026.
- [7] Treveil, M., et al. (2020), *Introducing MLOps*, O'Reilly Media.
- [8] Rahman, A. (2021), *DevSecOps : A Leader's Guide*, O'Reilly Media.